

Outils en ligne (utilisateur) : CNRTL

- **Flemm** : analyseur Flexionnel du français pour des corpus étiquetés
 - <https://www.cnrtl.fr/outils/flemm/>
- **Pompamo** : outil de détection de candidats à la néologie
 - <https://www.cnrtl.fr/outils/pompamo/>
- **DériF** : analyseur du lexique morphologiquement construit du Français
 - <https://www.cnrtl.fr/outils/DeriF/>
- **FastKwic** : indexation automatique permettant de produire un concordancier
 - <https://www.cnrtl.fr/outils/fastkwic/>

Outillage et logiciels du TAL

- Bibliothèque Python
- Outils
- plateformes d'entraînement de modèles

Bibliothèques généralistes

- **NLTK** : pédagogique, riche, mais lent
- **spaCy** : très performant, pratique, modèles entraînés, pipeline moderne
- **Stanza** (Stanford) : excellents modèles multilingues
- **HuggingFace Transformers** : modèles neuronaux pré-entraînés

SpaCy : <https://spacy.io/>

- une bibliothèque open-source
- particulièrement adaptée aux tâches du TAL
 - Prise en charge de plus de 75 langues
 - Pipelines préentraînées pour 25 langues, avec modèles spécifiques pour chaque langue
 - inclut POS tagging, parsing, NER, l'analyse en dépendances, la segmentation en phrases, la classification de texte, la lemmatisation, l'analyse morphologique, l'entity linking, etc
 - Utilisation de transformers préentraînés comme BERT pour des performances de pointe
 - <https://spacy.io/usage>

```
pip install spacy
python -m spacy download fr_core_news_sm
import spacy
nlp = spacy.load("fr_core_news_sm")
texte = "Marie Curie a reçu le prix Nobel de physique en 1903.«
doc = nlp(texte)
print("Token | Lemma | POS")
for token in doc:
    print(f"{token.text} | {token.lemma_} | {token.pos_}")
print("\nEntités nommées :")
for ent in doc.ents:
    print(f"{ent.text} ({ent.label_})")
```

Stanza

- bibliothèque open-source
- développée par le Stanford NLP Group
 - Prise en charge de plus de 75 langues avec des modèles préentraînés
 - Modèles spécifiques pour chaque langue, adaptés à la morphologie et à la syntaxe
 - Inclut : tokenisation, POS tagging, analyse syntaxique en dépendances, lemmatisation, REN
 - Modèles préentraînés basés sur (BiLSTM + CRF ou Transformers) deep learning
 - <https://stanfordnlp.github.io/stanza>

```
pip install stanza
import stanza
stanza.download("fr")
nlp = stanza.Pipeline("fr")
texte = "Marie Curie a reçu le prix Nobel de physique en 1903."
doc = nlp(texte)
print("Token | Lemma | POS")
for sentence in doc.sentences:
    for token in sentence.words:
        print(f"{token.text} | {token.lemma_} | {token.pos_}")
print("\nEntités nommées :")
for sentence in doc.sentences:
    for ent in sentence.ents:
        print(f"{ent.text} ({ent.type})")
```

NLTK Natural Language Toolkit

- bibliothèque Python open-source
- fournit une grande variété de ressources linguistiques et d'outils pour le traitement du texte
 - contient des corpus et outils principalement pour l'anglais, mais certaines ressources sont disponibles pour d'autres langues.
 - Fournit des corpus annotés, lexiques et modèles pré-entraînés
 - Inclut tokenization, POS tagging, Analyse syntaxique : grammaires contextuelles, parsing basé sur des règles ou probabilistes, REN, lemmatisation, racinisation
 - Corpora et lexiques : WordNet, Brown, Gutenberg, Stopwords, etc.
- Basée sur des algorithmes classiques de TAL (regex, HMM, arbres de décision, règles linguistiques)
- Moins orientée deep learning par défaut, mais peut être combinée avec des modèles externes.
- Plus lente que spaCy ou Stanza pour de grands corpus
 - <https://www.nltk.org>
 - <https://www.nltk.org/api/nltk.html>


```
pip install nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

texte = "Marie Curie a reçu le prix Nobel de physique en 1903.«
phrases = nltk.sent_tokenize(texte)

for phrase in phrases:
    mots = nltk.word_tokenize(phrase)
    pos_tags = nltk.pos_tag(mots)
    print("POS tags :", pos_tags)
    ne_tree = nltk.ne_chunk(pos_tags)
    print("Arbre NER :", ne_tree)
```

Sklearn

- fournit des modules et fonctions pour l'apprentissage automatique, notamment :
 - Régression, classification, clustering
 - Prétraitement des données
 - Sélection de modèles et validation croisée
 - Pipelines de traitement

#vectoriseur sac de mots

```
from sklearn.feature_extraction.text import CountVectorizer
```

#division train/test

```
from sklearn.model_selection import train_test_split
```

#Naive Baise, MultinomialNB = classe de scikit-learn qui implémente le classificateur Naive Bayes multinomial

#nomial = événement en math => multinomial = plusieurs événements discrets : ici le nombre de fois que chaque mot apparaît

```
from sklearn.naive_bayes import MultinomialNB
```

#mesurer le taux de bonnes prédictions

```
from sklearn.metrics import accuracy_score
```

#données

```
texts = ["J'adore ce film", "C'était horrible", "Très bon produit", "Je n'aime pas ce livre", "Super expérience", "Mauvais service"]
```

```
labels = ["positif", "negatif", "positif", "negatif", "positif", "negatif"]
```

Convertir le texte en vecteurs numériques

```
vectorizer = CountVectorizer()
```

```
#crée une instance de la classe CountVectorizer
```

```
X = vectorizer.fit_transform(texts)
```

```
#fit() et transform() sont des méthodes
```

```
# fit() : sert à apprendre quelque chose à partir des données, fit(texts) : construit le vocabulaire => ici il apprend le vocabulaire du corpus
```

```
#transform() : convertit chaque texte en un vecteur numérique basé sur ce vocabulaire
```

```
#fit_transform() : combine les deux étapes en une seule opération
```

Diviser en données d'entraînement et de test

```
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.33, random_state=42)
```

```
#train_test_split : fonction qui sert à diviser un jeu de données en deux parties
```

```
#X : les données d'entrée (features), ici la matrice numérique issue de CountVectorizer
```

```
#labels : les étiquettes/classes correspondantes à chaque donnée (positif / négatif)
```

```
#test_size=0.33 : 33% des données seront utilisées pour le test, le reste pour l'entraînement
```

```
#random_state=42 => fixe le hasard pour que la division soit toujours la même
```

```
#La fonction train_test_split divise les données de manière aléatoire.
```

```
#Sans préciser de random_state, à chaque exécution, la division pourrait être différente : les mêmes phrases pourraient se retrouver dans le train ou le test à chaque fois.
```

```
#Le nombre choisi pour random_state (42) n'a pas d'importance
```

Créer et entraîner un modèle Naive Bayes

```
model = MultinomialNB()
```

```
model.fit(X_train, y_train)
```

```
#model : un objet qui va apprendre à prédire des labels à partir des données
```

```
#fit() : entraîne le modèle sur les données d'entraînement
```

```
#Le modèle va apprendre les relations entre les features (X_train) et les labels (y_train)
```

Prédire et évaluer

```
y_pred = model.predict(X_test)
```

```
print("Prédictions :", y_pred)
```

```
print("Accuracy :", accuracy_score(y_test, y_pred))
```

Bibliothèques pour Deep learning

PyTorch

- bibliothèque Python open-source pour le deep learning
- développée par Facebook AI Research (FAIR)
- Permet de construire et entraîner des réseaux de neurones

Tensor Flow

- bibliothèque open-source pour le deep learning et le calcul numérique développé par Google Brain
- Permet de construire, entraîner et déployer des modèles de réseaux de neurones à grande échelle

Keras

- bibliothèque Python open-source pour le deep learning
- Initialement indépendante, Keras est maintenant intégrée directement à TensorFlow (tensorflow.keras)
- Permet de créer rapidement des réseaux de neurones complexes sans avoir à manipuler directement les tenseurs ou le graphe de calcul.

Outils : TreeTagger

- Lemmatiseur et étiqueteur morpho-syntaxique
- outil open-source développé par Helmut Schmid (Université de Stuttgart)
- ne fait pas d'analyse syntaxique ni de NER comme spaCy ou Stanza.
 - prend en charge plus de 20 langues
 - Pour chaque langue, des modèles préentraînés sont disponibles
 - Basé sur des modèles statistiques HMM (Hidden Markov Models)
 - Fonctionne sur Windows, Linux et macOS.
 - Utilisable via ligne de commande ou intégré dans des scripts
 - <http://www.cis.uni-muenchen.de/~schmid/tools/TreeTagger/>

- Télécharger TreeTagger : <http://www.cis.uni-muenchen.de/~schmid/tools/TreeTagger/>
- Installer le paquet Python treetaggerwrapper :
 - `pip install treetaggerwrapper`
- Télécharger le modèle français et décompressez-le dans le dossier TreeTagger

```
import treetaggerwrapper
```

```
# Chemin vers l'installation de TreeTagger et le modèle français
```

```
tagger = treetaggerwrapper.TreeTagger(TAGLANG='fr', TAGDIR='/chemin/vers/treetagger')
```

```
#TAGLANG='fr' : spécifie le modèle français
```

```
texte = "Marie Curie a reçu le prix Nobel de physique en 1903.«
```

```
tags = tagger.tag_text(texte)
```

```
#tagger.tag_text(texte) : retourne une liste de chaînes, chaque chaîne contient mot, lemme et POS séparés par des tabulations
```

```
print("Mot | Lemme | POS")
```

```
for t in tags:
```

```
# Chaque t est une chaîne "mot\tlemme\tPOS"
```

```
# Chaque t est une chaîne "mot\tlemme\tPOS"
```

```
    parts = t.split('\t')
```

```
    if len(parts) == 3:
```

```
        print(f"{parts[0]} | {parts[1]} | {parts[2]}")
```


Environnements de développement

- Jupyter notebooks
- Google Colab
- VS Code (Python)
- Outils de versioning (Git, GitHub/GitLab)

Jupyter Notebooks

- environnement interactif qui permet d'écrire et d'exécuter du code Python (et d'autres langages) directement dans un navigateur
 - Exécution de code **cellule par cellule**.
 - Possibilité d'insérer **texte, formules mathématiques (LaTeX) et visualisations**
 - Sauvegarde au format .ipynb pour partager le code et les résultats
 - Intégration avec des bibliothèques scientifiques : NumPy, pandas, matplotlib, spaCy, NLTK, etc.
 - <https://jupyter.org/documentation>

Google Colab

- Google Colaboratory est un service en ligne gratuit basé sur Jupyter Notebook, fourni par Google.
- Il permet d'exécuter du code Python dans le cloud, sans installation locale.
 - Exécution gratuite sur GPU et TPU pour des calculs intensifs
 - Partage facile via Google Drive et lien URL
 - Installation simple des bibliothèques Python : `!pip install ...`
 - Idéal pour le travail collaboratif
 - <https://colab.research.google.com/notebooks/intro.ipynb>

VS Code (Visual Studio Code)

- éditeur de code léger et puissant, utilisé pour le développement Python et TAL
 - Éditeur de texte avancé avec coloration syntaxique et complétion automatique
 - Extensions pour Python, Jupyter, etc.
 - Débogage intégré et gestion d'environnements virtuels
 - Support de projets complexes et fichiers multiples, contrairement aux notebooks
 - <https://code.visualstudio.com/docs>
 - Extension Python pour VS Code :
<https://marketplace.visualstudio.com/items?itemName=ms-python.python>

Outils de versioning (Git, GitHub/GitLab)

- Utiles pour gestion de projet informatique
- permettent de **suivre les modifications du code**, collaborer à plusieurs et gérer l'historique d'un projet.
 - **Git** : système de contrôle de version distribué
 - <https://git-scm.com/doc>
 - **GitHub / GitLab** : plateformes en ligne pour héberger des dépôts Git, collaborer et partager des projets
 - Gestion de versions (commits, branches, merges).
 - Collaboration multi-utilisateurs sur un même projet
 - Sauvegarde et partage du code
 - <https://docs.github.com> (plateforme collaborative)
 - <https://docs.gitlab.com> (plateforme collaborative alternative)

GitHub vs GitLab

- GitHub

- Plateforme de gestion de dépôts Git en ligne
- Hébergement de code, collaboration et partage open source
- Hébergement :
 - Principalement cloud (github.com)
 - Installation locale : GitHub Enterprise payant
 - Très large, surtout open source et projets publics

- GitLab

- Plateforme DevOps complète, suivi de projet et plus
- Gestion de projets de développement de bout en bout
- Hébergement :
 - Cloud (gitlab.com)
 - Installation locale : GitLab CE (Community Edition) gratuite et open-source pour serveur local.
 - Moins de visibilité grand public, mais très utilisé en entreprises

Outillage pour le traitement des corpus

- **AntConc** : concordancier
- **TXM** : analyse textuelle linguistique
- **Sketch Engine** (commercial, niveau pro)
- **NoSketch Engine** (open source)

Enjeux et limites

- Rapidité vs précision
- Modèles open-source vs API commerciales
- Limites des ressources (langues peu dotées)
- Importance des benchmarks et de la reproductibilité => toujours pertinent ?